Carlos Lopez and Kennedy Suon
CSC 3201 Section S10 Professor Yocam
Module 4: Hashing and Passwords

Task 1. SHA256 and the avalanche effect

The program hashes text with SHA256 and prints each digest in hexadecimal. For the bit change experiment, XOR with 1 changes only the lowest bit of the first input byte. The program counts differences in the input bits, digest bits and digest bytes.

Hello, world!

`315f5bdb76d078c43b8ac0064e4a0164612b1fce77c869345bfc94c75894edd3`

Python

`18885f27b5af9012df19e496460f9294d5ab76128824c6f993787004f6d9a7db`

Cryptography

`b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f86719f61857`

Input pair	Input bits changed	Digest bits changed	Digest bytes changed
security to recurity	1	123	32
password to qassword	1	144	32
Cryptography to Bryptography	1	133	32

Question 1. Changing one input bit changed many digest bits. All 32 digest bytes differed in each of the three trials. The bit counts were close to half of the 256 output bits, which is consistent with the avalanche effect. A changed byte can contain between one and eight changed bits.

Task 1. Collision measurements

The search keeps the first n bits of each SHA256 digest. A dictionary maps each shortened digest to its first input. A repeated digest with a different input is a collision. Inputs contain the digest size, batch number and counter, so the program never tests the same input twice within a search.

The dictionary is cleared after 8,000,000 entries to limit memory use. The next batch uses a different prefix. Counts and times include all completed batches. Collisions between separate batches are not detected, so the larger searches can take more attempts than a birthday search that retains every entry.

Bits	Inputs checked	Seconds	Cleared batches
8	30	0.000037	0
10	20	0.000025	0
12	92	0.000083	0
14	8	0.000014	0
16	206	0.000197	0
18	512	0.000462	0
20	2,805	0.003447	0
22	3,228	0.002869	0
24	4,629	0.005878	0
26	10,593	0.012494	0
28	7,327	0.011380	0
30	47,900	0.061909	0
32	140,346	0.170606	0
34	97,709	0.111948	0
36	238,111	0.432383	0
38	595,553	1.050686	0
40	1,558,826	2.150532	0
42	3,060,824	4.454515	0
44	4,810,293	6.940974	0
46	3,118,753	4.803426	0
48	96,742,068	156.185889	12
50	134,467,227	337.956984	16

The 50 bit collision used two distinct inputs:

```
50:16:6354586
50:16:6467226
Shared truncated value: 33bfcc28b3d09
```

Task 1. Graphs

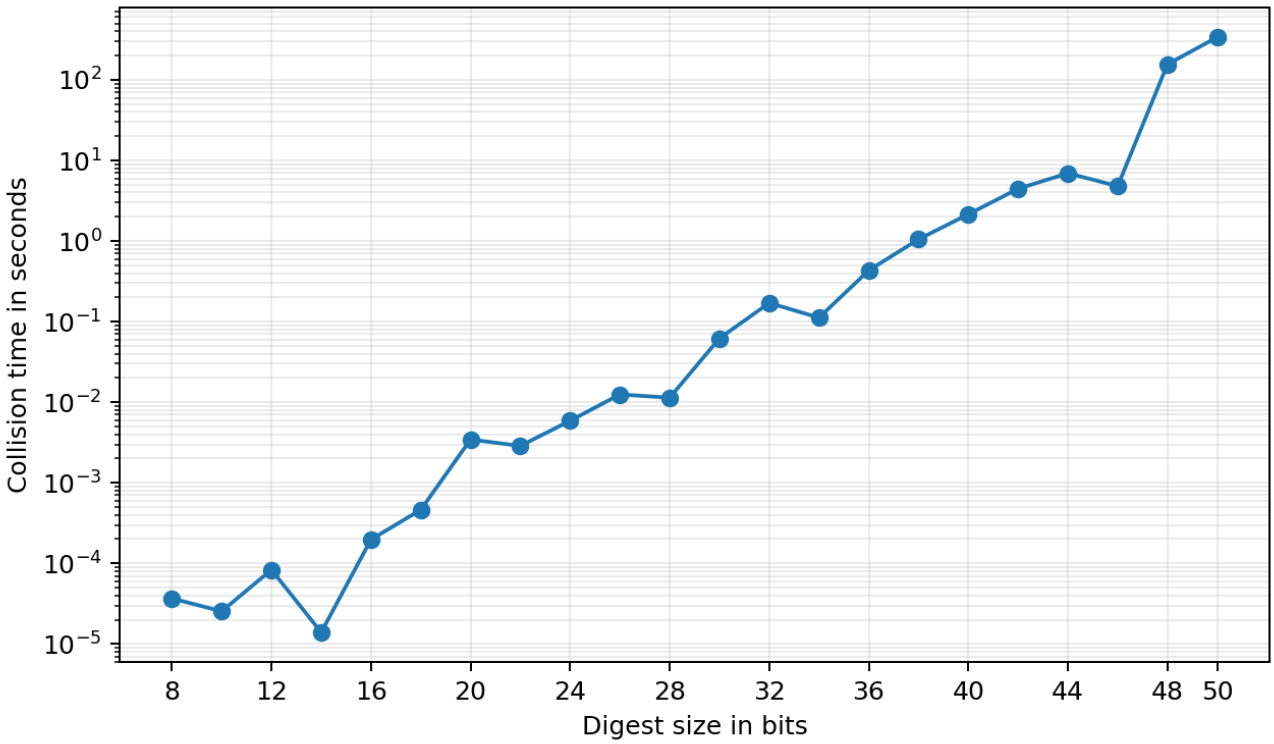


Figure 1. Digest size and measured collision time. The vertical axis uses a logarithmic scale.

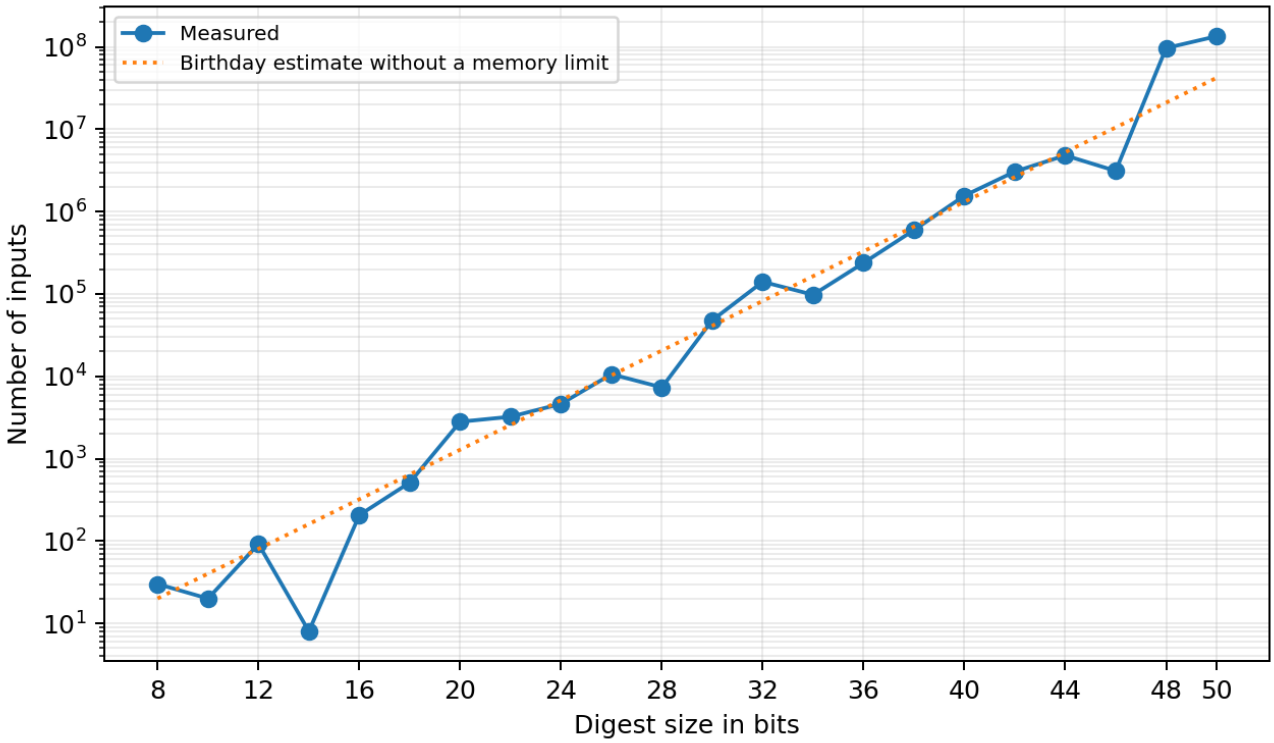


Figure 2. Digest size and inputs checked. The dotted line is the birthday estimate for a search that retains all earlier digests. The vertical axis uses a logarithmic scale.

Task 1. Questions 2 and 3

Question 2. An n bit digest has 2^n possible values. Hashing $2^n + 1$ distinct files guarantees that at least two have the same digest. This follows from the pigeonhole principle. Finding that guaranteed pair requires retaining enough information to compare the results.

For a hash behaving like a random function, a birthday search needs roughly $2^{n/2}$ hashes. A more precise expected count is 1.2533 times $2^{n/2}$. This is an average, not a guarantee for one run. Individual measurements can go down as the digest size increases.

The measurements generally required more work at larger digest sizes. The memory limit adds extra work when batches are cleared, so the upper measurements should not be treated as a direct test of the unlimited memory birthday estimate.

Across these searches, 244,907,060 inputs were tested in 514.36 seconds, or about 476,142 inputs per second. Using that rate as a rough throughput estimate, a full SHA256 birthday collision would need about 4.26 times 10^{38} hashes and about 2.84e+25 years. This extrapolation assumes constant speed and enough storage. Our bounded dictionary would be slower still. It is not a practical way to find a full SHA256 collision.

Question 3. An 8 bit digest has only 256 possible values, so finding a preimage for a chosen value is practical. Each independent trial has probability 1/256 of matching, giving an average of 256 trials. A collision only needs any pair to match and takes about 20 trials on average by the birthday estimate. A preimage is therefore harder than a collision, but both are easy at 8 bits. Trying 256 inputs does not guarantee a match to a particular target.

Digest pairs from Task 1b

security and recurity

```
5d2d3ceb7abe552344276d47d36a8175b7aeb250a9bf0bf00e850cd23ecf2e43
9991486255e076c8af6ade3013a512c747a643cdb4dfc3dde6914ad4dad3cf0b
```

password and qassword

```
5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
500753de1463102324a99774730af64ae33e82ca40569e618bfe157142900d1d
```

Cryptography and Bryptography

```
b584eec728548aced5a66c0267dd520a00871b5e7b735b2d8202f86719f61857
78caf0fdff16191040d7fd361d32cc247ec3c7ad235fc9299bf0fc2c62a86aa4
```

Task 2. Dictionary search

The program reads the classroom bcrypt hashes and searches unique NLTK words containing 6 through 10 letters. Brown corpus frequencies put common words first. Frequency ties use shorter words first through cost 11 and longer words first at costs 12 and 13. Every eligible word and its original case are preserved.

Hashes sharing an algorithm, cost and salt form a group, identified by their first 29 characters. Each candidate is hashed once per group with bcrypt.hashpw and compared with the unresolved hashes. Workers search separate dictionary chunks. bcrypt.checkpw verifies each recovered password.

Times and counts are cumulative for each group and recorded when the batch containing the match finishes. They include process startup and coordination. Shared work must not be added across users. Saved batches support resuming. Stopped time and unfinished interrupted batches are excluded.

Dictionary: 134,849 words. The search used 4 workers.

User	Cost	Password	Group hashes	Seconds
Bilbo	8	welcome	1,024	3.41
Gandalf	8	wizard	7,040	22.92
Thorin	8	diamond	4,096	13.37
Fili	9	desire	640	4.21
Kili	9	ossify	10,880	70.75
Balin	10	hangout	39,040	504.50
Dwalin	10	drossy	20,352	262.58
Oin	10	ispaghul	62,848	811.95
Gloin	11	oversave	66,432	1721.81
Dori	11	indoxylic	90,368	2337.39
Nori	11	swagsman	72,320	1872.65
Ori	12	airway	122,880	6309.50
Bifur	12	corrosible	22,272	1144.61
Bofur	12	libellate	58,496	3008.45
Durin	13	purrone	114,560	11758.45

Password position also affects recovery time. A higher cost password can be found quickly when it appears early. Question 4 compares hashing rates based on the actual candidate counts.

This offline attack finds a password whose hash matches. It does not decrypt bcrypt. Higher costs slow each guess. Different salts prevent reuse of a computed hash across users. The reused salts in this file allowed shared work, which real systems should prevent by assigning independent random salts.

Question 4. Larger password spaces

Let D be the number of eligible dictionary words and r be the measured number of bcrypt checks per second for one cost and worker count. Ordered word choices may repeat. The colon separators are fixed and add no choices. For a uniformly chosen password, the expected search time is N divided by twice r seconds. An exhaustive search takes N/r seconds, where N is the number of possible passwords.

For the numeric part, this calculation allows every digit string of length 1 through 5, including leading zeroes. That gives 111,110 possibilities. If only integers from 0 through 99,999 without leading zeroes are allowed, use 100,000 instead.

For this dictionary, D = 134,849. The possible passwords are:

Format	Candidates
word1:word2	18,184,252,801
word1:word2:word3	2,452,128,305,962,049
word1:word2:number	2,020,452,328,719,110

For each cost, r is the total candidate hashes divided by the completed group's elapsed seconds. The table gives average search times in years at that measured rate. One year is 365.25 days. An exhaustive search takes twice each listed average.

Cost	Hashes per second	Two words	Three words	Two words and digits
8	307.09	0.938	1.27e+05	1.04e+05
9	153.78	1.87	2.53e+05	2.08e+05
10	77.40	3.72	5.02e+05	4.14e+05
11	38.66	7.45	1e+06	8.28e+05
12	19.48	14.8	1.99e+06	1.64e+06
13	9.74	29.6	3.99e+06	3.29e+06

Increasing the bcrypt cost by one roughly doubles the hashing work. The measured rates also include program overhead and scheduling. They are estimates for this hardware and worker count, not universal rates.

These estimates assume uniform independent choices. Human word choices can be predictable, and another machine or bcrypt cost changes the rate.

Running the code

The supplied shadow.pdf file is included in the project folder. Use the VS Code terminal in that folder. Run each command separately.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m nltk.downloader words brown
python task1.py
python task2.py shadow.pdf --workers 4
```

These commands reuse saved progress when the run settings match. To start a new measurement run, add --fresh to the relevant task command. The largest collision searches and higher bcrypt costs may take a long time.

Both scripts save their results in the results folder. Task 1 also saves the two collision graphs. The shadow.pdf input is the classroom password file.

Packages in requirements.txt

```
bcrypt==5.0.0
nltk==3.10.3
matplotlib>=3.9,<4
pypdf>=6,<7
```

Sources

Eric Yocam. Cryptographic Hash Functions. Assignment instructions and questions.

Module 4 User Authentication and Module 4 User Authentication Assignment Hints. Password storage, dictionary attacks, bcrypt and parallel search.

Library documentation: Python hashlib and multiprocessing; NLTK Words and Brown corpora; bcrypt hashpw and checkpw.

Code: task1.py

```
"""SHA256 examples, avalanche measurements and truncated collisions."""

import argparse
import csv
import hashlib
import json
import math
import platform
import time
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "results"


def truncate(data, bits):
    # Keep the first n bits, including lengths that are not whole hex digits.
    return int.from_bytes(hashlib.sha256(data).digest()[:8], "big") >> (64 - bits)


def avalanche_examples():
    rows = []
    for original in [b"security", b"password", b"Cryptography"]:
        changed = bytes([original[0] ^ 1]) + original[1:]
        first = hashlib.sha256(original).digest()
        second = hashlib.sha256(changed).digest()
        row = {
            "input1": original.decode(), "input2": changed.decode(),
            "input_bits_different": sum((a ^ b).bit_count()
                                        for a, b in zip(original, changed)),
            "digest1": first.hex(), "digest2": second.hex(),
            "digest_bits_different": sum((a ^ b).bit_count()
                                         for a, b in zip(first, second)),
            "digest_bytes_different": sum(a != b for a, b in zip(first, second)),
        }
        assert row["input_bits_different"] == 1
        rows.append(row)
        print(json.dumps(row, indent=2), flush=True)
    return rows


def find_collision(bits, max_entries):
    start = time.perf_counter()
    attempts = 0
    batch = 0
    while True:
        seen = {}
        prefix = f"{bits}:{batch}:"
        for index in range(max_entries):
            message = f"{prefix}{index}"
            value = truncate(message.encode(), bits)
            attempts += 1
            if value in seen:
                first = f"{prefix}{seen[value]}"
                assert first != message
                return {
                    "bits": bits, "inputs": attempts,
                    "seconds": time.perf_counter() - start,
                    "input1": first, "input2": message,
                    "truncated_hex": f"{value:0{(bits + 3) // 4}x}",
                    "completed_batches": batch,
                }
            seen[value] = index
```

```
        # A new prefix keeps every input distinct after clearing the table.
        del seen
        batch += 1
        print(f"{bits} bits: {attempts:,} inputs checked", flush=True)


def save_results(data):
    OUT.mkdir(exist_ok=True)
    path = OUT / "task1_results.json"
    temporary = path.with_suffix(".tmp")
    temporary.write_text(json.dumps(data, indent=2) + "\n")
    temporary.replace(path)
    rows = data["collisions"]
    if not rows:
        return
    with (OUT / "collisions.csv").open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)


def plot_results(rows):
    bits = [r["bits"] for r in rows]
    for field, label, filename in [
        ("seconds", "Collision time in seconds", "collision_time.png"),
        ("inputs", "Number of inputs", "collision_inputs.png"),
    ]:
        fig, ax = plt.subplots(figsize=(7.2, 4.3))
        ax.plot(bits, [r[field] for r in rows], "o-", label="Measured")
        if field == "inputs":
            expected = [math.sqrt(math.pi / 2) * 2 ** (n / 2) for n in bits]
            ax.plot(bits, expected, ":", label="Birthday estimate without a memory limit")
            ax.legend(fontsize=8)
        ax.set(xlabel="Digest size in bits", ylabel=label, yscale="log")
        ax.set_xticks(bits[::2] + ([bits[-1]] if bits[-1] not in bits[::2] else []))
        ax.grid(alpha=0.25, which="both")
        fig.tight_layout()
        fig.savefig(OUT / filename, dpi=180)
        plt.close(fig)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--message", help="Print the SHA256 digest of this text and exit")
    parser.add_argument("--max-bits", type=int, default=50)
    parser.add_argument("--max-entries", type=int, default=8_000_000)
    parser.add_argument("--fresh", action="store_true", help="Replace earlier Task 1 results")
    args = parser.parse_args()
    if args.message is not None:
        print(hashlib.sha256(args.message.encode()).hexdigest())
        return
    if args.max_bits not in range(8, 51, 2) or args.max_entries < 2:
        parser.error("Use an even bit length from 8 to 50 and at least 2 entries")

    examples = []
    for message in ["Hello, world!", "Python", "Cryptography"]:
        digest = hashlib.sha256(message.encode()).hexdigest()
        print(f"{message}: {digest}")
        examples.append({"input": message, "digest": digest})
    avalanche = avalanche_examples()
    metadata = {"platform": platform.platform(), "python": platform.python_version(),
                "max_entries": args.max_entries}
    path = OUT / "task1_results.json"
    data = {"environment": metadata, "examples": examples,
            "avalanche": avalanche, "collisions": []}
    if path.exists() and not args.fresh:
        data = json.loads(path.read_text())
        if data["environment"] != metadata:
            parser.error("The environment or memory limit changed. Run with --fresh")
```

```
    completed = {r["bits"] for r in data["collisions"]}
    for bits in range(8, args.max_bits + 1, 2):
        if bits in completed:
            continue
        row = find_collision(bits, args.max_entries)
        data["collisions"].append(row)
        save_results(data)
        print(f"{bits} bits: collision after {row['inputs']:,} inputs "
              f"in {row['seconds']:.6f} seconds", flush=True)
    plot_results(data["collisions"])
    print(f"Results saved in {OUT}")


if __name__ == "__main__":
    main()
```

Code: task2.py

```
"""Search the supplied classroom bcrypt hashes with the NLTK words corpus."""

import argparse
import hashlib
import json
import multiprocessing as mp
import os
import platform
import re
import time
from collections import Counter
from pathlib import Path

import bcrypt

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "results"
CHUNK_SIZE = 32


def read_shadow(path):
    if path.suffix.lower() == ".pdf":
        from pypdf import PdfReader
        text = "\n".join(page.extract_text() or "" for page in PdfReader(path).pages)
    else:
        text = path.read_text()
    pattern = r"([\w.-]+)\s*:\s*(\$2[aby]\$\d{2}\$(?:[./A-Za-z0-9]\s*){53})"
    rows = []
    for name, value in re.findall(pattern, text):
        encoded = re.sub(r"\s+", "", value)
        if not 4 <= int(encoded[4:6]) <= 31:
            raise ValueError(f"Invalid bcrypt cost for {name}")
        if any(r["username"] == name for r in rows):
            raise ValueError(f"Duplicate username in the shadow file: {name}")
        rows.append({"username": name, "hash": encoded,
                     "algorithm": encoded.split("$")[1], "cost": int(encoded[4:6]),
                     "salt": encoded[7:29]})
    if not rows:
        raise ValueError("No complete username and bcrypt hash pairs were found")
    if len(re.findall(r"[\w.-]+\s*:\s*\$2", text)) != len(rows):
        raise ValueError("At least one bcrypt entry is incomplete or unsupported")
    return sorted(rows, key=lambda row: row["cost"])


def load_words():
    from nltk.corpus import brown, words
    eligible = list(dict.fromkeys(w for w in words.words()
                                  if 6 <= len(w) <= 10 and w.isascii() and w.isalpha()))
    frequency = Counter(w.lower() for w in brown.words())
    # Frequency only changes the search order. Password spelling stays unchanged.
```

```
    return sorted(eligible, key=lambda w: (-frequency[w.lower()], len(w), w))


def order_for_group(candidates, group):
    if group["cost"] < 12:
        return candidates
    from nltk.corpus import brown
    frequency = Counter(w.lower() for w in brown.words())
    # Keep saved work in place, then try longer words first within frequency ties.
    prefix = group.setdefault("original_order_prefix", group["next_index"])
    remaining = sorted(candidates[prefix:],
                       key=lambda w: (-frequency[w.lower()], -len(w), w))
    return candidates[:prefix] + remaining


def check_chunk(job):
    prefix, targets, candidates = job
    matches = {}
    for word in candidates:
        encoded = bcrypt.hashpw(word.encode(), prefix).decode()
        if encoded in targets:
            matches[encoded] = word
    return matches, len(candidates)


def save(data):
    OUT.mkdir(exist_ok=True)
    path = OUT / "task2_results.json"
    temporary = path.with_suffix(".tmp")
    temporary.write_text(json.dumps(data, indent=2) + "\n")
    temporary.replace(path)


def crack_group(group, candidates, workers, data):
    candidates = order_for_group(candidates, group)
    context = mp.get_context("spawn")
    rows = [r for r in data["users"] if r["hash"][:29] == group["prefix"]]
    targets = {r["hash"] for r in rows if r["status"] != "found"}
    width = workers * CHUNK_SIZE
    previous_seconds = group["seconds"]
    start = time.perf_counter()
    with context.Pool(workers) as pool:
        for offset in range(group["next_index"], len(candidates), width):
            end = min(offset + width, len(candidates))
            jobs = [(group["prefix"].encode(), targets,
                     candidates[i:min(i + CHUNK_SIZE, end)])
                    for i in range(offset, end, CHUNK_SIZE)]
            answers = pool.map(check_chunk, jobs)
            group["attempts"] += sum(count for _, count in answers)
            group["next_index"] = end
            group["seconds"] = previous_seconds + time.perf_counter() - start
            matches = {encoded: word for found, _ in answers for encoded, word in found.items()}
            for row in rows:
                if row["status"] == "found":
                    continue
                row["attempts"], row["seconds"] = group["attempts"], group["seconds"]
                row["status"] = "not found" if end == len(candidates) else "running"
                if row["hash"] in matches:
                    row["password"] = matches[row["hash"]]
                    assert bcrypt.checkpw(row["password"].encode(), row["hash"].encode())
                    row["status"] = "found"
                    targets.discard(row["hash"])
                    print(f"{row['username']}: {row['password']} in {row['seconds']:.2f} seconds",
                          flush=True)
            group["status"] = "found" if not targets else "running"
            if targets and end == len(candidates):
                group["status"] = "exhausted"
            save(data)
            print(f"Cost {group['cost']}: {group['attempts']:,} candidate hashes, "
```

```
                f"{group['seconds']:.2f} seconds, {len(targets)} hashes remaining", flush=True)
            if not targets:
                break


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("shadow", type=Path, help="The classroom shadow PDF or text file")
    parser.add_argument("--workers", type=int, default=min(4, os.cpu_count() or 1))
    parser.add_argument("--fresh", action="store_true", help="Replace earlier Task 2 results")
    args = parser.parse_args()
    if args.workers < 1:
        parser.error("Use at least one worker")
    try:
        targets = read_shadow(args.shadow)
        candidates = load_words()
    except LookupError:
        parser.error("Install the corpora with: python -m nltk.downloader words brown")
    except (OSError, ValueError) as error:
        parser.error(str(error))
    metadata = {
        "platform": platform.platform(), "python": platform.python_version(),
        "bcrypt": bcrypt.__version__, "workers": args.workers,
        "chunk_size": CHUNK_SIZE, "method": "shared salt groups",
        "dictionary_size": len(candidates),
        "dictionary_sha256": hashlib.sha256("\n".join(candidates).encode()).hexdigest(),
    }
    source_hash = hashlib.sha256(args.shadow.read_bytes()).hexdigest()
    data = {"environment": metadata, "source": args.shadow.name,
            "source_sha256": source_hash, "users": [], "groups": []}
    path = OUT / "task2_results.json"
    if path.exists() and not args.fresh:
        data = json.loads(path.read_text())
        if data["environment"] != metadata or data["source_sha256"] != source_hash:
            parser.error("The input or run settings changed. Run with --fresh")
    else:
        data["users"] = [{**target, "attempts": 0, "seconds": 0.0,
                          "password": None, "status": "pending"} for target in targets]
        for prefix in dict.fromkeys(r["hash"][:29] for r in targets):
            data["groups"].append({"prefix": prefix, "cost": int(prefix[4:6]),
                                   "next_index": 0, "attempts": 0, "seconds": 0.0,
                                   "status": "pending"})
    save(data)
    print(f"Dictionary: {len(candidates):,} unique words. Workers: {args.workers}", flush=True)
    try:
        for group in data["groups"]:
            if group["status"] in ["found", "exhausted"]:
                print(f"Saved cost {group['cost']}: {group['status']}", flush=True)
                continue
            crack_group(group, candidates, args.workers, data)
    except KeyboardInterrupt:
        print("Stopped. Run the same command to resume from the last saved batch.")
        return
    found_costs = {r["cost"] for r in data["users"] if r["status"] == "found"}
    missing = sorted(set(range(8, 14)) - found_costs)
    print(f"Results saved in {path}")
    remaining = [r["username"] for r in data["users"] if r["status"] != "found"]
    print(f"Verified passwords: {len(data['users']) - len(remaining)} of {len(data['users'])}")
    if remaining:
        print("Still missing: " + ", ".join(remaining))
    else:
        print("All supplied passwords were recovered.")
    if missing:
        print(f"Required costs still missing: {missing}")


if __name__ == "__main__":
    main()
```